

spectra-toolkit:

теория и разбор кода

Математика, хемометрика и Python для подбора каналов оптического сенсора

Октябрь 2026

Содержание

1	Как пользоваться конспектом	4
1.1	Путь данных через пакет	4
1.2	Какой раздел к какому модулю относится	4
1.3	Как читать	4
2	Python для чтения кода	4
2.1	Окружение и установка	5
2.2	Базовые типы и конструкции	5
2.3	Функции	5
2.4	Классы и объекты	6
2.5	Ошибки и предупреждения	6
2.6	Файлы, пути, регулярные выражения	7
2.7	NumPy: массивы	7
2.8	Pandas: таблицы	8
2.9	scikit-learn: единый интерфейс	8
2.10	argparse и pytest	9
3	Математика: минимум, который нужен	10
3.1	Векторы и матрицы	10
3.2	Среднее, центрирование, дисперсия	10
3.3	Метод наименьших квадратов и почему он не работает на спектрах	10
3.4	Сингулярное разложение (SVD) в двух словах	11
3.5	Комбинаторика	11

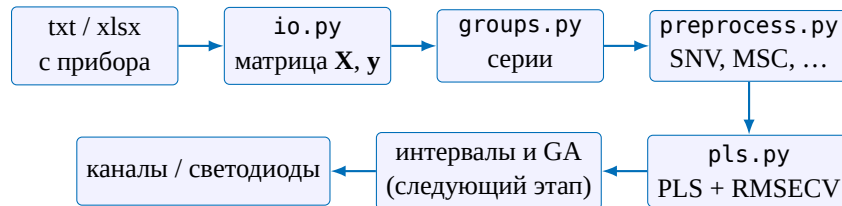
4	Спектры: что именно мы измеряем	11
4.1	Поглощение и закон Бугера--Ламберта--Бера	11
4.2	Ближний ИК и вода	12
4.3	Рассеяние	12
4.4	Матрица данных и её особенности	12
4.5	Образец, измерение, серия	12
4.6	Нормировка в вашем файле	13
5	Предобработка спектров	13
5.1	Зачем и общий принцип	13
5.2	Центрирование по среднему (mean-centering)	14
5.3	Min-max нормировка 0--1: два разных смысла	14
5.4	SNV (Standard Normal Variate)	14
5.5	MSC (Multiplicative Scatter Correction)	14
5.6	Сглаживание и производные Савицкого--Голея	15
5.7	Автошкалирование	15
5.8	Порядок шагов	15
6	PLS-регрессия	16
6.1	Зачем нужна и в чём идея	16
6.2	Модель	16
6.3	Алгоритм NIPALS для одной переменной y (PLS1)	17
6.4	Числовой пример, который можно проверить руками	17
6.5	Число латентных переменных A	18
6.6	Что делает PLSRegression	18
6.7	VIP: какие длины волн важны	18
7	Метрики и валидация модели	19
7.1	Ошибки и R^2	19
7.2	Схемы кросс-валидации	19
7.3	Как работает <code>cross_val_predict</code>	20
7.4	Смещение при отборе и вложенная валидация	20
8	Интервалы и генетический алгоритм	21
8.1	Постановка задачи	21
8.2	Усреднение по интервалам (понижение разрешения)	21

8.3	Ограничения из статьи (a)–(e)	21
8.4	Почему нужен не перебор, а оптимизация	22
8.5	Генетический алгоритм (GA)	22
8.6	Два цикла в статье	23
8.7	Что важно помнить на практике	23
9	PCA и PLS-DA (на будущее)	23
9.1	Метод главных компонент	23
9.2	PLS-DA для классификации	24
10	Построчный разбор кода	24
10.1	Предобработка (preprocess.py)	24
10.2	Метрики, кросс-валидация, PLS (pls.py)	27
10.3	Группы (groups.py)	30
10.4	Чтение и склейка файлов (io.py)	31
11	Ответы на вопросы	33
12	Глоссарий	35
13	Литература	35

1 Как пользоваться конспектом

Цель этого текста --- чтобы вы могли *объяснить каждую строку* пакета `spectra-toolkit`: что она делает, зачем она нужна и какой математике соответствует. Конспект устроен «снизу вверх»: сначала инструменты (Python, NumPy, scikit-learn), потом математика, потом хеометрика (предобработка, PLS, валидация), потом подбор каналов по статье Галянина и, наконец, построчный разбор настоящего кода.

1.1 Путь данных через пакет



1.2 Какой раздел к какому модулю относится

Модуль	Что делает	Теория в разделах
<code>io.py</code>	читает файлы, собирает матрицу X	2, 3, 4
<code>groups.py</code>	определяет серии для кросс-валидации	2, 7
<code>preprocess.py</code>	SNV, MSC, Савицкий--Голей, min-max	3, 5
<code>pls.py</code>	PLS, RMSEC/RMSECV, сравнение предобработок	3, 6, 7
<code>cli.py</code>	команды <code>spectra merge</code> , <code>spectra benchmark</code>	2
(будет) интервалы, GA	подбор каналов по Галянину	8
(будет) PCA, PLS-DA	анализ главных компонент, классификация	9

1.3 Как читать

В каждом разделе есть рамки четырёх видов: **идея** (суть в двух словах), **пример** (маленькие числа, которые можно проверить руками), **ловушка** (где легко ошибиться) и **проверь себя** (вопросы; ответы в конце). Числовые примеры проверены кодом. Раздел 10 разбирает реальные функции из пакета построчно, его удобно держать открытым рядом с PyCharm.

Если мало времени, минимальный путь: разделы 3.1--3.3, 5, 6.1--6.5, 7, затем 10.

2 Python, который нужен, чтобы читать код

Здесь только то, что реально встречается в пакете. Каждый фрагмент можно набрать в консоли PyCharm (*Python Console*) и посмотреть результат.

2.1 Окружение и установка

- **Виртуальное окружение** (`.venv`) --- отдельная папка с копией Python и библиотек для одного проекта, чтобы версии не конфликтовали. PyCharm создаёт её при открытии проекта.
- `pip install -e ".[dev]"` --- установить пакет *в режиме разработки* (-e, editable): Python не копирует код, а ссылается на папку `src/spectra`, поэтому правки видны сразу. Часть `[dev]` добавляет необязательные зависимости (здесь `pytest`), их перечисляет `pyproject.toml`.
- `pyproject.toml` --- «паспорт» пакета: имя, версия, зависимости, команда `spectra`, которая запускает `spectra.cli:main`.
- Структура `src/spectra/`: папка с файлом `__init__.py` становится *пакетом*. В нём `from .io import Dataset` означает «импортировать из соседнего модуля `io.py`» (точка = текущий пакет).

2.2 Базовые типы и конструкции

```
names = ["a_1", "b_1"]           # список (list): изменяемый, упорядоченный
pair = (1, 2)                     # кортеж (tuple): неизменяемый
d = {"snv": 1, "msc": 2}          # словарь (dict): ключ -> значение
s = {"interpolate", "raise"}      # множество (set): быстрая проверка "x in s"

squares = [i**2 for i in range(5)] # list comprehension: [0, 1, 4, 9, 16]
for i, name in enumerate(names):   # enumerate даёт пары (номер, элемент)
    print(i, name)
for a, b in zip([1, 2], ["x", "y"]): # zip склеивает последовательности
    print(a, b)

f"RMSECV = {6.2772:.2f}"           # f-строка: подстановка значения, .2f = 2 знака
x = "да" if 5 > 3 else "нет"       # условное выражение
first = next((t for t in [3, 7] if t > 5), None) # первый подходящий или None
```

Последняя строка встречается в `infer_groups`. Выражение в скобках --- *генератор*: оно выдаёт элементы по одному, а `next(генератор, значение_по_умолчанию)` берёт первый подходящий и останавливается. Если подходящих нет, вернётся `None` (значение «ничего»).

```
a, b = 3, 4                       # распаковка: a=3, b=4
a, b = b, a                       # обмен значений
name, params = ("snv", {})        # распаковка кортежа
{"label": "x", **{"LV": 6}}       # ** вставляет пары словаря: {"label": "x", "LV": 6}
```

2.3 Функции

```
def rmse(y_true, y_pred) -> float: # -> float: подсказка типа результата
    return float(...)              # return отдаёт значение

def scan(X, y, groups=None, max_lv: int = 15, cv: str = "logo"):
    ...
scan(X, y)                         # groups=None, max_lv=15, cv="logo" (значения по умолчанию)
scan(X, y, cv="loo")               # именованный аргумент
```

- **Подсказки типов** (`x: int, -> float`) ничего не проверяют при запуске, это документация для вас и для PyCharm. Строка `from __future__ import annotations` в начале модулей позволяет писать такие подсказки современным синтаксисом (`List[str]`, `A | None`).
- **Строка документации** (docstring) --- текст в тройных кавычках сразу после `def` или `class`; показывается во всплывающей подсказке.
- `make_steps(**params)` и `REGISTRY[name](**params)`: **словарь распаковывает словарь в именованные аргументы. Если `params = {"deriv": 1}`, то `SavGol(**params)` то же самое, что `SavGol(deriv=1)`.

2.4 Классы и объекты

Класс --- шаблон объекта с данными (атрибутами) и поведением (методами).

```
class Counter:
    def __init__(self, start=0):      # конструктор: вызывается при Counter(...)
        self.value = start           # self = этот конкретный объект
    def add(self, k):
        self.value += k
        return self                  # вернуть сам объект позволяет цепочки

c = Counter(10); c.add(5); print(c.value)  # 15
```

- **Наследование**: `class SNV(BaseEstimator, TransformerMixin)` --- класс SNV получает всё, что умеют родители. Это даёт ему совместимость со scikit-learn (раздел 2.9).
- **Датакласс**: `@dataclass class Dataset` с аннотированными полями автоматически получает `__init__`, `__repr__`. Метод `__post_init__` вызывается сразу после создания, в нём мы приводим типы и проверяем размеры.
- **Атрибуты с подчёркиванием в конце** (`reference_`) --- соглашение scikit-learn: «значение появилось после fit».
- **Декоратор @dataclass** --- функция, которая берёт класс и дополняет его.

2.5 Ошибки и предупреждения

```
try:
    xs.append(float(tokens[0]))      # может упасть
except (ValueError, IndexError):    # ловим только ожидаемые ошибки
    continue                         # пропустить эту строку файла

raise ValueError("текст")           # сами сообщаем об ошибке
import warnings; warnings.warn("...") # предупреждение: работа продолжается
```

Принцип проекта: лучше сразу упасть с понятным сообщением (неверная сетка длин волн, пропуски в данных), чем тихо посчитать неверное число.

2.6 Файлы, пути, регулярные выражения

- `Path("dir/file.txt")` (модуль `pathlib`) --- объект пути. Полезно: `.stem` (имя без расширения), `.suffix` (расширение), `.read_bytes()`, `Path(d).glob("*.txt")` (все подходящие файлы).
- **Байты и кодировки:** файл прибора читается как байты и затем `decode("utf-8-sig")` превращает их в текст; если не вышло, пробуем `cp1251` (старая русская Windows-кодировка).
- **Регулярные выражения** (модуль `re`):

```
import re
re.fullmatch(r"\d+", "10")      # совпало: строка целиком из цифр
re.fullmatch(r"\d+", "0.069")   # None: есть точка
re.search(r"[;\t ]", "1,5 2,5") # есть ли ; табуляция или пробел -> да
re.split(r"[;\t ]+", "a b;c")   # ['a','b','c']: разрезать по любым из этих знаков
```

Префикс `r""` (raw) нужен, чтобы `\d` не считалось специальной последовательностью Python.

2.7 NumPy: массивы

Массив (`ndarray`) --- таблица чисел одного типа. Это основа всех вычислений.

```
import numpy as np
X = np.array([[1., 2., 3.],
              [4., 5., 6.]])      # 2 строки (образцы) x 3 столбца (длины волн)
X.shape      # (2, 3)
X.T          # транспонирование, форма (3, 2)
X[0]         # первая строка: [1,2,3]
X[:, 1]      # второй столбец: [2,5]
X[:, 1:]     # все строки, столбцы начиная со второго
X[X > 3]     # булева маска: [4,5,6]
m = (wl >= 900) & (wl <= 1700)   # маска по диапазону; & = "и" (not 'and'!)
X[:, m]      # оставить только столбцы, где m == True
```

Оси. `axis=0` --- «вдоль строк» (результат по каждому столбцу), `axis=1` --- «вдоль столбцов» (результат по каждой строке).

```
X.mean(axis=0) # [2.5, 3.5, 4.5] среднее по образцам для каждой длины волны
X.mean(axis=1) # [2.0, 5.0]      среднее по спектру для каждого образца
X.mean(axis=1, keepdims=True)   # [[2.],[5.]], форма (2,1), а не (2,)
```

Транслирование (broadcasting). Операция между массивами разной формы «растягивает» меньший:

```
X - X.mean(axis=1, keepdims=True)
# (2,3) - (2,1) -> из каждой строки вычитается её среднее.
# Без keepdims=True форма (2,) была бы согласована со столбцами (3 != 2) и дала бы ошибку/неверный
# результат.
```

Ловушка

`keepdims=True` --- причина, по которой в SNV, MSC и min-max стоят именно такие вызовы. Если забыть, вычитание пойдёт не вдоль того направления, и ошибки не будет, если матрица квадратная.

Прочее, что встречается в коде:

- `A @ B` --- матричное произведение (внутренние размеры совпадают: $(n \times p)(p \times k)$).
- `np.asarray(X, dtype=float)` --- привести к массиву чисел с плавающей точкой (не копирует, если уже подходит); `np.empty_like(X)` --- пустой массив той же формы.
- `sd[sd == 0] = 1.0` --- присвоение по маске: везде, где $sd = 0$, поставить 1 (защита от деления на ноль).
- `np.ravel(a)` --- «вытянуть» в одномерный массив (нужно, потому что `PLSRegression.predict` возвращает форму $(n, 1)$).
- `np.polyfit(x, y, 1)` --- подгонка прямой методом наименьших квадратов, возвращает [наклон, свободный_член].
- `np.interp(x_new, x, a)` --- линейная интерполяция; `np.argsort(x)` --- индексы, упорядочивающие x (интерполяции нужны возрастающие x).
- `np.allclose(a, b, atol=1e-6)` --- равны ли массивы с допуском (числа с плавающей точкой нельзя сравнивать через `==`).
- `np.isnan`, `np.nan` --- «не число» (пропуск).
- `np.sqrt`, `np.mean`, `np.sum` --- понятно из названий; `**2` --- возведение в квадрат.

2.8 Pandas: таблицы

`DataFrame` --- таблица с подписанными строками и столбцами. В пакете она нужна для чтения Excel и для вывода результатов.

```
import pandas as pd
raw = pd.read_excel("data.xlsx", header=None) # header=None: первая строка - данные, не заголовок
raw.iloc[0, 2:] # iloc = доступ по номерам: строка 0, столбцы с 2-го
raw.drop(index=0) # таблица без строки 0
pd.to_numeric(s, errors="coerce") # превратить в числа; что не получилось -> NaN
df["RMSECV"].idxmin() # индекс строки с минимальным значением
df.loc[idx, "LV"] # loc = доступ по подписям
df.sort_values("RMSECV").reset_index(drop=True)
pd.DataFrame({"wavelength": wl, "s1": a}) # таблица из словаря столбцов
df.to_csv("out.csv", index=False)
```

2.9 scikit-learn: единый интерфейс

Объекты `scikit-learn` делятся на *оценщики* (estimators) с общими методами:

Метод	Смысл
<code>fit(X, y)</code>	«обучиться» на данных: запомнить параметры (среднее, веса модели)
<code>transform(X)</code>	применить обученное преобразование (для предобработки)
<code>predict(X)</code>	выдать предсказание (для моделей)
<code>fit_transform(X)</code>	<code>fit</code> затем <code>transform</code>

- Наследование от `TransformerMixin` даёт `fit_transform` автоматически, а от `BaseEstimator` --- методы `get_params/set_params`, нужные для клонирования. Поэтому в `__init__` параметры сохраняются *под теми же именами*: `self.window = window`.
- **Pipeline**: цепочка `[("snv", SNV()), ("pls", PLSRegression(...))]`. При `fit` данные последовательно проходят через `fit_transform` каждого шага, последний шаг обучается. При `predict` все шаги применяют только `transform`. Так параметры предобработки считаются *только* по обучающим данным.
- **cross_val_predict(model, X, y, groups, cv)**: для каждого фолда клонирует модель, обучает на обучающей части, предсказывает проверочную; возвращает предсказания для всех образцов (каждый предсказан моделью, которая его не видела).
- **Разбиватели**: `LeaveOneOut`, `LeaveOneGroupOut`, `GroupKFold`; метод `split(X, y, groups)` выдаёт пары индексов (обучение, проверка).

2.10 argparse и pytest

- **argparse** разбирает командную строку: `add_argument("xlsx")` --- обязательный позиционный аргумент, `--cv` --- именованный с значением по умолчанию, `add_subparsers` --- подкоманды (`merge`, `benchmark`), `set_defaults(func=...)` --- привязывает функцию к подкоманде.
- `if __name__ == "__main__":` --- код под ним выполняется только при запуске файла как программы, но не при импорте.
- **pytest** находит файлы `test_*.py` и функции `test_*`; `assert условие` --- проверка; фикстура `tmp_path` даёт временную папку. Тест, который падает, указывает на ошибку в коде (или в самом тесте, как у нас однажды было).

Проверь себя

1. Что вернёт `X.mean(axis=1)` для матрицы 80×465 и какую форму будет иметь результат с `keepdims=True`?
2. Почему `a == b` для чисел с плавающей точкой лучше заменить на `np.allclose`?
3. Что произойдёт, если в классе-трансформере записать параметр в `__init__` под другим именем атрибута?
4. Чем отличаются `loc` и `iloc`?

3 Математика: минимум, который нужен

3.1 Векторы и матрицы

Один спектр --- это вектор из p чисел (поглощение на p длинах волн). Все n спектров образуют матрицу $\mathbf{X} \in \mathbb{R}^{n \times p}$: строка = образец, столбец = длина волны. В нашем файле $n = 80$, $p = 465$. Влажность образцов --- вектор $\mathbf{y} \in \mathbb{R}^n$.

- **Транспонирование:** $\mathbf{X}^\top$ имеет форму $p \times n$; элемент $(\mathbf{X}^\top)_{ji} = x_{ij}$.
- **Скалярное произведение** векторов: $\mathbf{a}^\top \mathbf{b} = \sum_j a_j b_j$. **Норма:** $\|\mathbf{a}\| = \sqrt{\mathbf{a}^\top \mathbf{a}}$. Вектор длины 1 называется единичным.
- **Произведение матриц:** $(\mathbf{AB})_{ik} = \sum_j a_{ij} b_{jk}$; допустимо, если число столбцов $\mathbf{A}$ равно числу строк $\mathbf{B}$. Форма: $(n \times p)(p \times k) = (n \times k)$. Это $\mathbf{A} @ \mathbf{B}$.
- $\mathbf{X}\mathbf{w}$ для вектора $\mathbf{w} \in \mathbb{R}^p$ даёт вектор длины n : каждый образец превращается в одно число $t_i = \sum_j x_{ij} w_j$, то есть *взвешенную сумму* по длинам волн. В PLS это «счёт» (score) образца.
- $\mathbf{X}^\top \mathbf{y}$ --- вектор длины p , j -й элемент равен $\sum_i x_{ij} y_i$: насколько j -й столбец «движется вместе» с $\mathbf{y}$.
- **Ранг** матрицы --- число линейно независимых строк (или столбцов). $\text{rank } \mathbf{X} \leq \min(n, p)$.

3.2 Среднее, центрирование, дисперсия

Среднее вектора: $\bar{a} = \frac{1}{n} \sum_i a_i$. **Центрирование** --- вычитание среднего: $a_i - \bar{a}$. Для матрицы $\mathbf{X}$ среднее считают по столбцам (средний спектр $\bar{\mathbf{x}} \in \mathbb{R}^p$), и $\mathbf{X}_c = \mathbf{X} - \mathbf{1}\bar{\mathbf{x}}^\top$, где $\mathbf{1}$ --- столбец из n единиц. В коде: $\mathbf{X} - \mathbf{X}.\text{mean}(\text{axis}=0)$.

Дисперсия и стандартное отклонение (SD):

$$s^2 = \frac{1}{n - \text{ddof}} \sum_{i=1}^n (a_i - \bar{a})^2, \quad s = \sqrt{s^2}.$$

Параметр ddof (delta degrees of freedom) равен 0 для «генеральной» дисперсии и 1 для «выборочной» (несмещённой оценки). Это важно: в NumPy std по умолчанию использует ddof = 0, а в нашем SNV стоит ddof = 1; StandardScaler из scikit-learn использует 0.

Среднее и SD

Вектор (1, 2, 3, 6): $\bar{a} = 3$; отклонения (-2, -1, 0, 3); сумма квадратов $4 + 1 + 0 + 9 = 14$. При ddof = 1: $s^2 = 14/3 = 4,667$, $s = 2,160$. При ddof = 0: $s^2 = 3,5$, $s = 1,871$.

Ковариация двух векторов: $\text{cov}(a, b) = \frac{1}{n-1} \sum_i (a_i - \bar{a})(b_i - \bar{b})$. **Корреляция** --- та же величина, нормированная на SD: $r = \text{cov}(a, b) / (s_a s_b) \in [-1, 1]$. Для центрированных векторов ковариация пропорциональна скалярному произведению $\mathbf{a}^\top \mathbf{b}$. Эту связь использует PLS.

3.3 Метод наименьших квадратов и почему он не работает на спектрах

Линейная модель: $\hat{\mathbf{y}} = \bar{\mathbf{y}} + \mathbf{X}_c \mathbf{b}$. Метод наименьших квадратов (МНК) ищет $\mathbf{b}$, минимизирующий сумму квадратов ошибок:

$$\min_{\mathbf{b}} \|\mathbf{y}_c - \mathbf{X}_c \mathbf{b}\|^2 \Rightarrow \mathbf{b} = (\mathbf{X}_c^\top \mathbf{X}_c)^{-1} \mathbf{X}_c^\top \mathbf{y}_c.$$

Чтобы существовала обратная матрица $(\mathbf{X}_c^\top \mathbf{X}_c)^{-1}$ размера $p \times p$, она должна иметь полный ранг p . Но $\text{rank}(\mathbf{X}_c^\top \mathbf{X}_c) \leq n - 1$ (центрирование забирает одну степень свободы), а у нас $n - 1 = 79 < 465 = p$. Поэтому МНК *не определён*: бесконечно много $\mathbf{b}$ дают нулевую ошибку на обучении.

Вторая проблема --- **мультиколлинеарность**: соседние длины волн почти одинаковы, столбцы $\mathbf{X}$ сильно коррелируют, и даже при $n > p$ матрица $\mathbf{X}^\top \mathbf{X}$ почти вырождена, а $\mathbf{b}$ получается нестабильным (маленький шум в данных, огромные изменения коэффициентов).

Идея

Спектр из 465 чисел на самом деле содержит лишь несколько независимых «направлений» изменчивости (концентрация воды, рассеяние, температура). Методы проекции (PCR, PLS) сначала находят эти немногие направления (латентные переменные), а потом строят регрессию уже на них.

3.4 Сингулярное разложение (SVD) в двух словах

Любую матрицу можно записать как $\mathbf{X}_c = \mathbf{U}\mathbf{S}\mathbf{V}^\top$, где столбцы $\mathbf{V}$ (ортонормированные) --- главные направления в пространстве длин волн, $\mathbf{S}$ --- диагональная матрица неотрицательных чисел $s_1 \geq s_2 \geq \dots$ (сингулярные значения), столбцы $\mathbf{U}$ --- ортонормированные «координаты» образцов. Первое направление $\mathbf{v}_1$ --- то, вдоль которого образцы разбросаны сильнее всего. Эта конструкция лежит в основе PCA (раздел 9); ею же объясняется ранг: число ненулевых s_k равно рангу.

3.5 Комбинаторика

Число способов выбрать K позиций из M без учёта порядка:

$$\binom{M}{K} = \frac{M!}{K!(M-K)!}.$$

Для $M = 465$ позиций интервалов (по числу длин волн) получаем $\binom{465}{3} \approx 1,7 \cdot 10^7$, $\binom{465}{5} \approx 1,8 \cdot 10^{11}$, $\binom{465}{7} \approx 8,9 \cdot 10^{14}$. Число растёт как $M^K/K!$, поэтому полный перебор при $K = 7$ невозможен. Подробнее в разделе 8.

Проверь себя

1. Какую форму имеет $\mathbf{X}^\top \mathbf{X}$ при $n = 80$, $p = 465$? Какой у неё максимально возможный ранг?
2. Что такое «центрировать» матрицу $\mathbf{X}$ и вдоль какой оси считается среднее?
3. Почему при $\text{ddof} = 1$ SD больше, чем при $\text{ddof} = 0$?

4 Спектры: что именно мы измеряем

4.1 Поглощение и закон Бугера--Ламберта--Бера

Свет интенсивностью I_0 проходит через образец и ослабляется до I . Пропускание $T = I/I_0$. Поглощение (оптическая плотность):

$$A = -\log_{10} T = \varepsilon c l,$$

где ε --- коэффициент поглощения вещества на данной длине волны, c --- концентрация, l --- длина пути. В идеальном случае поглощения разных веществ складываются: $A(\lambda) = \sum_k \varepsilon_k(\lambda) c_k l$. Именно поэтому линейная модель (PLS) вообще применима: спектр смеси --- линейная комбинация спектров компонентов. Отклонения от линейности дают рассеяние, температурные эффекты, сильное поглощение (насыщение детектора).

4.2 Ближний ИК и вода

В диапазоне 900--1700 нм видны обертоны и составные полосы колебаний О--Н, С--Н, N--Н. Вода даёт доминирующую полосу около 1450 нм (в статье про мочу описана как комбинационная полоса с участием валентных колебаний ν_1 и ν_3), более слабые полосы лежат около 970 и 1190 нм. Для определения *влажности* это и есть информативная область, но интенсивная полоса 1450 нм у толстых образцов может насыщаться, поэтому слабые полосы тоже полезны. Форма и положение полос воды зависят от температуры и водородных связей (идея аквафотомии).

4.3 Рассеяние

Реальные образцы (ткани, порошки, биоимплантаты) рассеивают свет. Это добавляет к спектру:

- **аддитивный** сдвиг (спектр целиком смещён вверх/вниз);
- **мультипликативный** множитель (спектр растянут или сжат из-за изменения длины пути).

SNV и MSC (раздел 5) предназначены для устранения именно этих эффектов. Но если рассеяние само несёт информацию (в статье про молоко определяется жир по рассеянию на частицах), то такая «коррекция» вредна. Поэтому предобработку *сравнивают* на ваших данных, а не назначают заранее.

4.4 Матрица данных и её особенности

- **Строка X_i** . --- один образец (усреднённый из повторных измерений); **столбец** --- одна длина волны.
- Шаг сетки в вашем файле 1,658 нм, ширина реальных полос --- десятки нм, поэтому соседние столбцы почти одинаковы (коллинеарность).
- $p \gg n$ (465 против 80): моделей «слишком много» для данных, нужны проекционные методы и строгая валидация.

4.5 Образец, измерение, серия

Три разных понятия, которые легко смешать:

- **Измерение** (replicate): один замер; вы измеряете каждый образец 3 раза, вынимая и возвращая его в прибор, а потом усредняете.
- **Образец** (точка калибровки): один усреднённый спектр с известной влажностью y .
- **Серия**: ряд точек, полученных от *одного физического куска* на разных стадиях сушки. Влажность в серии меняется от 70 % до 0.

Точки внутри серии очень похожи (тот же кусок, та же геометрия, то же рассеяние). Если одни из них попадут в обучение, а соседние --- в проверку, модель «узнает» кусок, а не влажность, и оценка ошибки будет слишком оптимистичной. Поэтому кросс-валидацию делают **по группам** (раздел 7), а `infer_groups` определяет группу по имени образца.

4.6 Нормировка в вашем файле

Все значения в `data_for_training_from_article.xlsx` лежат между 0 и 1, а максимум каждого спектра близок к 1 (около полосы 1450 нм). Это похоже на деление каждого спектра на свой максимум (в коде `MaxSpectrum`). Такая нормировка убирает информацию об абсолютной интенсивности, зато делает спектры сопоставимыми по масштабу. Если у вас сохранились исходные (ненормированные) спектры, стоит сравнить результаты на них.

Проверь себя

1. Почему аддитивность поглощений оправдывает линейную модель?
2. Чем «измерение» отличается от «серии» и почему для кросс-валидации важна именно серия?
3. Что произойдёт с информацией об абсолютной интенсивности после деления спектра на его максимум?

5 Предобработка спектров

5.1 Зачем и общий принцип

Предобработка --- преобразование $X \rightarrow \tilde{X}$ до построения модели. Цели: убрать мешающие эффекты (рассеяние, дрейф базовой линии, шум), выровнять масштабы, подчеркнуть полезные признаки (производные). Важно разделять два типа преобразований:

Тип	Примеры	Что «обучается» на данных
Построчные (каждый спектр сам по себе)	SNV, min-max по спектру, деление на максимум, Савицкий--Голей	Ничего: параметры берутся из самого спектра
По столбцам (между образцами)	центрирование, min-max по длине волны, автошкалирование, MSC (эталон)	Среднее, min/max, SD или эталонный спектр <i>по обучающей выборке</i>

Утечка данных (data leakage)

Если вычислить среднее/минимум/эталон по *всем* 80 образцам и только потом делить на обучение и проверку, проверочные образцы незаметно повлияли на модель, и ошибка кросс-валидации получится заниженной. В пакете все шаги стоят внутри `Pipeline`, а он при каждом фолде заново вызывает `fit` только на обучающей части. Тест `test_no_leakage_minmax_feature_fit_only_on_train` это проверяет.

5.2 Центрирование по среднему (mean-centering)

$\tilde{x}_{ij} = x_{ij} - \bar{x}_j$, где $\bar{x}_j$ --- среднее по образцам для длины волны j . Модель после этого описывает отклонения от среднего спектра. То же делается с y . В нашем пакете центрирование выполняет сам PLSRegression, отдельного шага нет: цепочка «только центрирование» --- это пустой список `[]` (поэтому в таблице бенчмарка есть строка `centering only`). Именно так обработаны спектры в статье про молоко.

5.3 Min-max нормировка 0--1: два разных смысла

По каждому спектру (MinMaxSpectrum): $\tilde{x}_{ij} = \frac{x_{ij} - \min_k x_{ik}}{\max_k x_{ik} - \min_k x_{ik}}$. Каждый спектр растягивается в диапазон $[0, 1]$. Параметры берутся из самого спектра.

По каждой длине волны (MinMaxScaler, имя `minmax_feature`): $\tilde{x}_{ij} = \frac{x_{ij} - \min_k x_{kj}}{\max_k x_{kj} - \min_k x_{kj}}$, где $\min$ и $\max$ берутся по образцам для столбца j . Параметры обучаются на обучающей выборке и применяются к проверочной (она может выйти за $[0, 1]$, это нормально).

Min-max по спектру

$$(2, 4, 10) \rightarrow \left(\frac{2-2}{8}, \frac{4-2}{8}, \frac{10-2}{8}\right) = (0, 0,25, 1).$$

Деление на максимум (MaxSpectrum): $\tilde{x}_{ij} = x_{ij} / \max_k |x_{ik}|$; так выглядит нормировка вашего файла.

5.4 SNV (Standard Normal Variate)

Каждый спектр центрируется на своё среднее и делится на своё SD:

$$\tilde{x}_{ij} = \frac{x_{ij} - \bar{x}_i}{s_i}, \quad \bar{x}_i = \frac{1}{p} \sum_j x_{ij}, \quad s_i = \sqrt{\frac{1}{p-1} \sum_j (x_{ij} - \bar{x}_i)^2}.$$

Усреднение здесь идёт *вдоль спектра* (по j), поэтому `axis=1`. Убирает аддитивный сдвиг (вычитание среднего) и мультипликативный масштаб (деление на SD).

SNV

Спектр из четырёх точек (1, 2, 3, 6): среднее 3, $s = 2,160$ (раздел 3.2). Результат (-0,926, -0,463, 0, 1,389). Среднее результата 0, SD равно 1.

5.5 MSC (Multiplicative Scatter Correction)

Идея: каждый спектр $\mathbf{x}_i$ есть «эталонный спектр» $\mathbf{x}_{\text{ref}}$ (обычно средний по обучающей выборке), растянутый и сдвинутый эффектами рассеяния. Подгоняем прямую МНК:

$$x_{ij} \approx a_i + b_i x_{\text{ref},j} \quad (\text{по } j), \quad \tilde{x}_{ij} = \frac{x_{ij} - a_i}{b_i}.$$

В коде `np.polyfit(reference, row, 1)` возвращает (b_i, a_i) (наклон, затем свободный член); `reference_` --- эталон, запомненный в `fit`.

MSC

Эталон (1, 2, 3), спектр (3, 5, 7) = $1 + 2 \cdot (1, 2, 3)$: $a = 1, b = 2$, результат $(3 - 1, 5 - 1, 7 - 1)/2 = (1, 2, 3)$, то есть спектр совпал с эталоном.

В отличие от SNV, MSC использует информацию из *других* образцов (эталон), поэтому его нужно обучать только на обучающей части.

5.6 Сглаживание и производные Савицкого--Голея

Берётся окно из $2m + 1$ соседних точек (window), в нём методом наименьших квадратов подгоняется полином степени q (polyorder), а значением в центре окна объявляется значение полинома (сглаживание) или его производной порядка d (deriv). Оказывается, результат всегда есть свёртка спектра с фиксированным набором коэффициентов c_k :

$$\tilde{x}_j = \sum_{k=-m}^m c_k x_{j+k},$$

где c зависит только от m, q, d . Окно должно быть нечётным и больше q .

Коэффициенты, проверено через `scipy.signal.savgol_coeffs`

Окно 5, полином степени 2: сглаживание $\frac{1}{35}(-3, 12, 17, 12, -3)$; первая производная $\frac{1}{10}(-2, -1, 0, 1, 2)$.

Ловушка

В коде `savgol_filter` вызывается без параметра `delta`, поэтому производная считается по номеру точки, а не по нанометру. Чтобы получить единицы «на нм», её нужно разделить на $\Delta\lambda^d$ (у вас $\Delta\lambda \approx 1,658$ нм). Для PLS масштаб не важен, но если сравнивать величины в разных единицах --- важно. Производная первого порядка убирает аддитивный сдвиг, второго --- и линейный наклон, но сильно усиливает шум, поэтому её применяют вместе со сглаживанием окна 11--21 точек.

5.7 Автошкалирование

$\tilde{x}_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j}$ по каждому столбцу. Выравнивает вклад всех длин волн. Для спектров обычно не лучший выбор, потому что усиливает шумные, малоинформативные участки; в нашем бенчмарке он оказался в числе хороших, но разница в пределах разброса. `StandardScaler` использует `ddof = 0`.

5.8 Порядок шагов

Цепочка применяется слева направо: `["snv", {"savgol": {"deriv": 1}}]` означает «сначала SNV, потом производная». Порядок меняет результат (SNV нелинейна по данным, а производная линейна). Типичные комбинации в литературе: сглаживание или производная $\rightarrow$ SNV; MSC или SNV $\rightarrow$ центрирование (его делает PLS).

Проверь себя

1. Почему SNV не нужно «обучать» на обучающей выборке, а MSC нужно?
2. Чем min-max по спектру отличается от min-max по длине волны и какой из них в вашем файле уже сделан (с поправкой на деление на максимум)?
3. Посчитайте руками SNV для спектра (2, 4, 6) при ddof = 1.
4. Зачем нужна производная первого порядка и чем она рискует?

6 PLS-регрессия

6.1 Зачем нужна и в чём идея

Мы выяснили (раздел 3.3): обычный МНК на спектрах не работает. Есть два проекционных метода.

- **PCR** (регрессия на главные компоненты): сначала PCA находит направления наибольшей изменчивости $\mathbf{X}$, потом регрессия на них. Недостаток: самое изменчивое направление в $\mathbf{X}$ (например, рассеяние) может не иметь отношения к $\mathbf{y}$.
- **PLS** (partial least squares, проекция на латентные структуры): направления выбираются так, чтобы они одновременно объясняли изменчивость $\mathbf{X}$ и были как можно сильнее связаны с $\mathbf{y}$.

Идея

PLS строит несколько «латентных переменных» $t_1, t_2, \dots, t_A$. Каждая --- это взвешенная сумма длин волн, $t = \mathbf{X}\mathbf{w}$, причём веса $\mathbf{w}$ подобраны так, чтобы ковариация t с $\mathbf{y}$ была максимальной. Затем $\mathbf{y}$ регрессируется на эти немногие t (обычно $A = 3 \dots 10$ вместо 465 исходных переменных).

6.2 Модель

Для центрированных $\mathbf{X}_c, \mathbf{y}_c$ (в разделе используем те же обозначения $\mathbf{X}, \mathbf{y}$):

$$\mathbf{X} = \mathbf{T}\mathbf{P}^\top + \mathbf{E}, \quad \mathbf{y} = \mathbf{T}\mathbf{q} + \mathbf{f},$$

Объект	Форма	Название и смысл
$\mathbf{T}$	$n \times A$	<i>scores</i> : координаты образцов в пространстве латентных переменных
$\mathbf{W}$	$p \times A$	<i>weights</i> : по ним строится $\mathbf{T} = \mathbf{X}\mathbf{W}^*$ (веса на длинах волн)
$\mathbf{P}$	$p \times A$	<i>loadings</i> по $\mathbf{X}$: как латентные переменные «восстанавливают» спектр
$\mathbf{q}$	$A \times 1$	коэффициенты регрессии $\mathbf{y}$ на $\mathbf{T}$
$\mathbf{E}, \mathbf{f}$	---	остатки (то, что модель не объяснила)

Здесь $\mathbf{W}^* = \mathbf{W}(\mathbf{P}^\top \mathbf{W})^{-1}$ --- «пересчитанные» веса, с которыми счета получаются прямо из исходной $\mathbf{X}$. Итоговый регрессионный вектор в исходных переменных:

$$\mathbf{b} = \mathbf{W}(\mathbf{P}^\top \mathbf{W})^{-1} \mathbf{q} \in \mathbb{R}^p, \quad \hat{y} = \bar{y} + (\mathbf{x} - \bar{\mathbf{x}})^\top \mathbf{b}.$$

Это *спектр коэффициентов*: положительные значения на длине волны λ означают, что рост поглощения на λ увеличивает предсказанную влажность. Его график показывает, какие участки спектра использует модель.

6.3 Алгоритм NIPALS для одной переменной y (PLS1)

Пусть $\mathbf{X}^{(1)} = \mathbf{X}_c$, $\mathbf{y}^{(1)} = \mathbf{y}_c$. Для $a = 1, \dots, A$:

1. **Вес:** $\mathbf{w}_a = \frac{\mathbf{X}^{(a)\top} \mathbf{y}^{(a)}}{\|\mathbf{X}^{(a)\top} \mathbf{y}^{(a)}\|}$.
2. **Счёт:** $\mathbf{t}_a = \mathbf{X}^{(a)} \mathbf{w}_a$.
3. **Нагрузка по X:** $\mathbf{p}_a = \frac{\mathbf{X}^{(a)\top} \mathbf{t}_a}{\mathbf{t}_a^\top \mathbf{t}_a}$.
4. **Коэффициент по y:** $q_a = \frac{\mathbf{y}^{(a)\top} \mathbf{t}_a}{\mathbf{t}_a^\top \mathbf{t}_a}$.
5. **Вычитание (deflation):** $\mathbf{X}^{(a+1)} = \mathbf{X}^{(a)} - \mathbf{t}_a \mathbf{p}_a^\top$, $\mathbf{y}^{(a+1)} = \mathbf{y}^{(a)} - q_a \mathbf{t}_a$.

Почему шаг 1 такой? Мы хотим выбрать единичный вектор $\mathbf{w}$, максимизирующий ковариацию счёта $\mathbf{t} = \mathbf{X}\mathbf{w}$ с $\mathbf{y}$, то есть $\mathbf{w}^\top \mathbf{X}^\top \mathbf{y}$. По неравенству Коши–Буняковского максимум достигается, когда $\mathbf{w}$ направлен вдоль $\mathbf{X}^\top \mathbf{y}$. После шагов 2–5 из данных вычитается всё, что объяснила первая латентная переменная, и следующая ищет *оставшуюся* связь. Поэтому счёта $\mathbf{t}_a$ получаются взаимно ортогональными.

6.4 Числовой пример, который можно проверить руками

Четыре образца, две длины волны

$$\mathbf{X} = \begin{pmatrix} 1 & 2 \\ 2 & 1 \\ 3 & 4 \\ 4 & 3 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} 1 \\ 2 \\ 3 \\ 5 \end{pmatrix}.$$

Средние: $\bar{\mathbf{x}} = (2,5; 2,5)$, $\bar{y} = 2,75$. После центрирования:

$$\mathbf{X}_c = \begin{pmatrix} -1,5 & -0,5 \\ -0,5 & -1,5 \\ 0,5 & 1,5 \\ 1,5 & 0,5 \end{pmatrix}, \quad \mathbf{y}_c = \begin{pmatrix} -1,75 \\ -0,75 \\ 0,25 \\ 2,25 \end{pmatrix}.$$

Компонента 1. $\mathbf{X}_c^\top \mathbf{y}_c = (6,5; 3,5)$, норма $\sqrt{54,5} = 7,382$, поэтому $\mathbf{w}_1 = (0,880; 0,474)$. Счёт $\mathbf{t}_1 = \mathbf{X}_c \mathbf{w}_1 = (-1,558; -1,151; 1,151; 1,558)$. $\mathbf{t}_1^\top \mathbf{t}_1 = 7,505$, $\mathbf{p}_1 = (0,776; 0,668)$, $q_1 = 0,984$. Заметьте, что $\mathbf{p}_1^\top \mathbf{w}_1 = 1$: для первой компоненты так всегда.

Тогда $\mathbf{b}^{(1)} = \mathbf{w}_1 q_1 / (\mathbf{p}_1^\top \mathbf{w}_1) = (0,866; 0,466)$, предсказания $\hat{y} = 2,75 + \mathbf{X}_c \mathbf{b}^{(1)} = (1,218; 1,617; 3,883; 4,282)$.

Компонента 2. После вычитания: $\mathbf{w}_2 = (0,474; -0,880)$ (ортогонален $\mathbf{w}_1$), $q_2 = 0,762$. Итоговый вектор коэффициентов с двумя компонентами

$$\mathbf{b}^{(2)} = \mathbf{W}(\mathbf{P}^\top \mathbf{W})^{-1} \mathbf{q} = (1,375; -0,125),$$

предсказания $(0,75; 2,25; 3,25; 4,75)$. Это в точности решение МНК: $\mathbf{b} = (\mathbf{X}_c^\top \mathbf{X}_c)^{-1} \mathbf{X}_c^\top \mathbf{y}_c = \frac{1}{16} \begin{pmatrix} 5 & -3 \\ -3 & 5 \end{pmatrix} (6,5; 3,5)^\top = (1,375; -0,125)$.

Вывод примера: при $A = p$ PLS совпадает с обычной регрессией, а при $A < p$ даёт «сжатую» версию. Все числа получены в коде и совпадают с `PLSRegression(2, scale=False)` из `scikit-learn`.

6.5 Число латентных переменных A

- Слишком мало --- модель недоучена (большая ошибка и на обучении, и на проверке).
- Слишком много --- модель подгоняется под шум: RMSEC продолжает падать, а RMSECV сначала падает, потом растёт.

Выбирают A по минимуму RMSECV (функция `best_lv`) либо по правилу «наименьшая модель, у которой RMSECV отличается от минимума меньше чем на стандартную ошибку». Максимум A ограничен: не больше $\min(n_{\text{train}} - 1, p)$ (в `scan_components` это строка с `min_train`). В статье Галянина число LV для сенсора берётся из полноспектральной модели и не растёт, потому что линейное преобразование (усреднение интервалов) не увеличивает ранг данных.

6.6 Что делает `PLSRegression`

- `n_components` --- число латентных переменных A .
- `scale=False` --- не делить на SD. Центрирование $\mathbf{X}$ и $\mathbf{y}$ всё равно выполняется внутри. При `scale=True` модель автошкалирует $\mathbf{X}$ и $\mathbf{y}$, что для спектров обычно нежелательно (шаг автошкалирования мы включаем явно, чтобы управлять им).
- `fit(X, y)` обучает; `predict(X)` возвращает массив формы $(n, 1)$; атрибут `coef_` --- вектор $\mathbf{b}$.

6.7 VIP: какие длины волн важны

Variable Importance in Projection: для переменной j

$$\text{VIP}_j = \sqrt{p \frac{\sum_{a=1}^A \text{SS}_a (w_{ja} / \|\mathbf{w}_a\|)^2}{\sum_{a=1}^A \text{SS}_a}}, \quad \text{SS}_a = \mathbf{q}_a^2 \mathbf{t}_a^\top \mathbf{t}_a.$$

Средний квадрат VIP равен 1, поэтому порог $\text{VIP} > 1$ считается «важной» переменной. Именно так в статье про мочу была выбрана область 1300--1600 нм. Для нашего выбора каналов VIP полезен как подсказка, где искать, но окончательный отбор делает GA.

Проверь себя

1. Что максимизирует PLS на каждом шаге и чем это отличается от PCA?
2. Почему последовательные $\mathbf{t}_a$ ортогональны?
3. Что произойдёт с RMSEC и RMSECV при росте числа латентных переменных?
4. Что показывает график вектора $\mathbf{b}$ по длинам волн?

7 Метрики и валидация модели

7.1 Ошибки и R^2

Для набора из k образцов с известными y_i и предсказаниями $\hat{y}_i$:

$$\text{RMSE} = \sqrt{\frac{1}{k} \sum_{i=1}^k (\hat{y}_i - y_i)^2}, \quad R^2 = 1 - \frac{\sum_i (\hat{y}_i - y_i)^2}{\sum_i (y_i - \bar{y})^2}.$$

Одна и та же формула называется по-разному в зависимости от того, на каких данных считается:

Название	Данные	Смысл
RMSEC (калибровка)	обучающая выборка, модель обучена на ней же	насколько хорошо модель подгоняет данные; всегда оптимистична
RMSECV (кросс-валидация)	предсказания для образцов, которых модель при обучении не видела (по фолдам)	оценка ошибки на новых данных при небольшой выборке
RMSEP (предсказание)	отдельная проверочная/тестовая выборка	честная оценка ошибки на новых данных

R^2 интерпретируется как доля дисперсии y , объяснённая моделью. Она равна 1 при идеальных предсказаниях, 0 --- если модель не лучше простого среднего $\bar{y}$, и *может быть отрицательной*, если модель хуже среднего (в статье про мочу так было для 10 из 18 анализов).

Руками

$y = (1, 2, 3, 4)$, $\hat{y} = (1,5; 1,5; 3,5; 3,5)$. Ошибки $\pm 0,5$, сумма квадратов $4 \cdot 0,25 = 1$, $\text{RMSE} = \sqrt{1/4} = 0,5$. Отклонения от среднего 2,5: $(-1,5; -0,5; 0,5; 1,5)$, сумма квадратов 5. $R^2 = 1 - 1/5 = 0,8$.

Замечание о делителе. Наши `gmse` делит на k , как формула (1) в статье Галянина. Иногда RMSEC делят на $n - A - 1$ (учёт числа параметров модели); для сравнения с чужими работами это нужно помнить.

7.2 Схемы кросс-валидации

Идея: разбить образцы на части (фолды), по очереди обучать на всех фолдах, кроме одного, и предсказывать оставленный. Затем объединить предсказания и посчитать RMSE по всем образцам. Это даёт RMSECV.

Схема (в коде)	Как делится	Когда применять
<code>loo</code> (leave-one-out)	по одному образцу на фолд, n фолдов	независимые образцы; у нас даёт завышенную точность
<code>logo</code> (leave-one-group-out)	один фолд = одна группа целиком	образцы сгруппированы (серии, повторы); рекомендуется
<code>groupkfold5</code>	5 фолдов, группы не разрываются	больше групп, нужна скорость

В статье Галянина: CV1 --- leave-one-out, CV2 --- сегментная (по группам повторных измерений одного образца). Результат: для жира $RMSE_{CV1} = 0,116$, $RMSE_{CV2} = 0,125$ --- сегментная чуть больше, потому что строже.

Что показали ваши данные

Полноспектральная PLS без предобработки (только центрирование): LOO даёт $RMSECV \approx 5,0$ при 10 LV, $R_{CV}^2 = 0,944$; leave-one-series-out даёт $RMSECV \approx 6,5$ при 6 LV, $R_{CV}^2 = 0,906$. Разница --- «подсматривание» соседей по серии в LOO.

7.3 Как работает `cross_val_predict`

1. Разбиватель (LeaveOneGroupOut) выдаёт для каждого фолда индексы обучающих и проверочных образцов.
2. Для каждого фолда берётся *чистая копия* модели (Pipeline целиком!), `fit` на обучающей части, `predict` на проверочной.
3. Предсказания всех фолдов собираются в один вектор в порядке исходных образцов.
4. `rmse(y, y_cv)` считает RMSECV по всем образцам сразу.

Копия Pipeline включает предобработку, поэтому среднее/эталон MSC считаются заново на каждом фолде. Это и есть защита от утечки.

7.4 Смещение при отборе и вложенная валидация

Если мы *выбираем* что-то (число LV, предобработку, интервалы) по минимуму RMSECV, то сам этот минимум становится оптимистичной оценкой: из многих вариантов мы взяли тот, которому «повезло» на этих данных. Чем больше вариантов перебираем (а GA перебирает огромное число комбинаций), тем сильнее эффект. Защита:

- отложить **внешнюю тестовую выборку** (целые серии), которая не участвует ни в отборе, ни в подборе LV, и дать RMSEP на ней;
- **вложенная CV**: внутренний цикл выбирает, внешний оценивает (в статье про мочу: 5-кратная внешняя CV и 3-кратная внутренняя для гиперпараметров);
- **пермутационный тест**: перемешать y много раз, повторить весь отбор и посмотреть, как часто случайные метки дают такую же ошибку (в той же статье 1000 перестановок).

С 11 группами в вашем файле выбор каналов лучше проверять отдельной серией, которую в отборе не использовали.

Проверь себя

1. Почему RMSEC почти всегда меньше RMSECV?
2. Что вернёт R^2 , если все предсказания равны среднему y ?
3. Почему при группах LOO даёт слишком хорошую оценку?
4. Почему нельзя считать RMSECV минимального варианта окончательной точностью после перебора миллионов комбинаций каналов?

8 Подбор каналов сенсора: интервалы и генетический алгоритм

Этот раздел описывает метод из статьи Галянина, Мелентьевой и Богомолова (2015), который мы будем реализовывать следующим шагом. Кода пока нет, поэтому здесь только теория и план.

8.1 Постановка задачи

Сенсор на светодиодах (LED) не измеряет весь спектр. Каждый светодиод излучает в узкой полосе, а детектор интегрирует свет, прошедший через образец. Нужно выбрать K светодиодов (центры и ширины полос) так, чтобы PLS-модель по K числам предсказывала влажность почвы так же точно, как по всему спектру. Тогда полноспектральная модель служит эталоном (benchmark).

8.2 Усреднение по интервалам (понижение разрешения)

Канал k задаётся весами $w_{kj} \geq 0$ на длинах волн $j = 1, \dots, p$ (вне полосы веса равны 0). Значение канала для образца i --- взвешенное среднее поглощения:

$$z_{ik} = \frac{\sum_j w_{kj} x_{ij}}{\sum_j w_{kj}} \iff \mathbf{Z} = \mathbf{X} \mathbf{W}, \quad W_{jk} = \frac{w_{kj}}{\sum_{j'} w_{kj'}}.$$

Здесь $\mathbf{W}$ --- матрица размера $p \times K$, а $\mathbf{Z}$ --- новая матрица данных $n \times K$. PLS затем строится на $\mathbf{Z}$ вместо $\mathbf{X}$. Три способа задать веса:

- **Прямоугольный интервал:** $w_{kj} = 1$ внутри интервала шириной Δ вокруг центра c_k , иначе 0 (простое среднее).
- **Гауссовы веса:** $w_{kj} = \exp(-(\lambda_j - c_k)^2 / (2\sigma^2))$ --- приближение формы реального светодиода (в статье дисперсия 16 нм, ширина окна 71 нм).
- **Спектры реальных светодиодов:** $w_{kj} = E_k(\lambda_j)$, где E_k --- измеренный спектр излучения; тогда ГА выбирает не центры, а *светодиоды из каталога*.

Так как $\mathbf{Z} = \mathbf{X} \mathbf{W}$ --- линейное преобразование, ранг данных не вырастает: число латентных переменных не больше, чем в полноспектральной модели (в статье для жира 4--5, для белка 3--4), а число каналов K должно быть не меньше числа LV.

Физическое допущение

Детектор измеряет *интенсивность*, а мы усредняем *поглощение* $A = -\log_{10} T$. Среднее от логарифма не равно логарифму среднего: $-\log \bar{T} \neq \overline{-\log T}$. Для узких полос и малых A разница мала, для широких полос или больших A --- заметна. Статья принимает это упрощение. В проекте имеет смысл потом проверить, не меняется ли результат, если усреднять пропускание $T = 10^{-A}$ и логарифмировать после усреднения.

8.3 Ограничения из статьи (a)–(e)

- Фиксированные ширины интервалов.
- Фиксированные позиции центров (остаётся выбирать только комбинацию).

- (с) Минимальное расстояние между соседними интервалами (чтобы не было двух почти одинаковых светодиодов).
- (d) Весовая функция известна (например, гауссиана).
- (е) Веса индивидуальны: берутся спектры светодиодов из каталога.

Ограничения (а)--(с) резко уменьшают пространство поиска.

8.4 Почему нужен не перебор, а оптимизация

Число способов выбрать K позиций из M равно $\binom{M}{K}$. Для $M = 465$ (все длины волн вашего файла как возможные центры):

K	$\binom{465}{K}$
3	$1,7 \cdot 10^7$
5	$1,8 \cdot 10^{11}$
7	$8,9 \cdot 10^{14}$

Если строить 390 PLS-моделей в секунду (скорость из статьи на их процессоре), то для $K = 7$ перебор займёт $8,9 \cdot 10^{14} / 390 \approx 2,3 \cdot 10^{12}$ с, то есть около $7 \cdot 10^4$ лет. Кросс-валидация по группам умножит стоимость ещё в число фолдов. В статье полный перебор возможен только после жёстких ограничений: число испытаний снижено с $1,6 \cdot 10^{16}$ до $8,3 \cdot 10^6$.

8.5 Генетический алгоритм (GA)

GA --- эвристика, имитирующая эволюцию. Она не гарантирует глобальный оптимум, но находит хорошие решения быстро (в статье 8--49 минут против 322--368 минут перебора с ограничениями, при сопоставимой или лучшей ошибке).

Термин GA	В нашей задаче
Хромосома (особь)	Вектор из K позиций интервалов, например индексы центров $(c_1, \dots, c_K)$
Ген	Одна позиция c_k
Популяция	Набор из N особей (случайные комбинации)
Приспособленность (fitness)	Качество модели: чем меньше ошибка PLS на $\mathbf{Z}$, тем лучше
Отбор	Лучшие особи чаще становятся «родителями» (турнир или рулетка)
Скрещивание (crossover)	Потомок берёт часть генов от одного родителя и часть от другого
Мутация	Случайный сдвиг одного гена (позиции) на небольшую величину
Элитизм	Лучшая особь копируется в следующее поколение без изменений
Критерий остановки	Улучшение ниже порога (10^{-5} в статье) или лимит поколений

Схема одного поколения:

1. Для каждой особи собрать $\mathbf{Z} = \mathbf{XW}$ и обучить PLS.

2. Посчитать приспособленность.
3. Отобрать родителей, скрестить, мутировать, получить новое поколение.
4. Повторять, пока улучшение не станет пренебрежимо малым.

8.6 Два цикла в статье

- **Внутренний цикл** (эволюция одной популяции): целевая функция --- RMSEC, потому что она считается быстро (одна подгонка вместо n или числа групп).
- **Внешний цикл**: берётся несколько популяций (разные начальные приближения), и лучшая выбирается по RMSECV₂ (кросс-валидация по группам). Так оценка по RMSEC не приводит к переобучению, а дорогая кросс-валидация считается только по лучшим решениям.

8.7 Что важно помнить на практике

- **Воспроизводимость**: GA случаен. Фиксируйте seed и запускайте несколько раз; устойчивые каналы (повторяющиеся в разных запусках) надёжнее.
- **Смещение отбора**: см. раздел 7.4. Нужна контрольная серия, не участвовавшая в отборе.
- **Число каналов**: растите K и смотрите, где ошибка перестаёт заметно падать (в статье 7 светодиодов; больше --- без выигрыша).
- **Физические ограничения**: у вас диапазон 900--1700 нм, поэтому нужны InGaAs-детектор и ИК-светодиоды (каталоги производителей), а не кремниевые 400--1100 нм из статьи.

Проверь себя

1. Запишите $\mathbf{Z} = \mathbf{X}\mathbf{W}$ для двух каналов и трёх длин волн с весами (1, 1, 0) и (0, 1, 1) (как выглядят $\mathbf{W}$ и z_{i1}, z_{i2} ?).
2. Почему число LV в сенсорной модели не больше, чем в полноспектральной?
3. Зачем во внутреннем цикле используется RMSEC, а не RMSECV?
4. Какие операторы GA сохраняют хорошие решения, а какие добавляют разнообразие?

9 PCA и PLS-DA (на будущее)

9.1 Метод главных компонент

PCA находит ортогональные направления, вдоль которых изменчивость спектров максимальна. Для центрированной $\mathbf{X}_c$ берём SVD $\mathbf{X}_c = \mathbf{U}\mathbf{S}\mathbf{V}^T$ (раздел 3.4):

- **нагрузки** (loadings): столбцы $\mathbf{V}$, это «спектры» главных компонент; их график показывает, какие длины волн определяют компоненту;
- **счета** (scores): $\mathbf{T} = \mathbf{X}_c\mathbf{V} = \mathbf{U}\mathbf{S}$, координаты образцов; график PC1 против PC2 показывает группировку и выбросы;
- **доля объяснённой дисперсии**: $\frac{s_k^2}{\sum_l s_l^2}$ для компоненты k .

Пример из статьи про мочу: PC1 объясняет 79,6 % дисперсии, и это не рак, а общая концентрация/разбавление мочи. Вывод для практики: самая большая компонента не обязательно полезна для интересующего вас вопроса. Поэтому PCA используют для контроля качества данных, поиска выбросов (расстояние Махаланобиса в пространстве первых компонент) и понимания источников изменчивости, а регрессию строят отдельно.

В scikit-learn: `PCA(n_components=5).fit(X)`, затем `transform(X)` (счета), `components_` (нагрузки), `explained_variance_ratio_`.

9.2 PLS-DA для классификации

PLS-DA --- это тот же PLS, но y кодирует класс (0 и 1, для нескольких классов --- несколько столбцов). Предсказание сравнивают с порогом (часто 0,5). Для несбалансированных классов используют **сбалансированную точность**:

$$\text{БАсс} = \frac{1}{2} (\text{чувствительность} + \text{специфичность}).$$

Наш интерфейс моделей (`fit/predict`) позволит добавить PLS-DA без переделки кросс-валидации; группы по-прежнему нужны, чтобы повторные образцы одного объекта не оказывались в обучении и проверке одновременно.

Проверь себя

1. Чем счета PCA отличаются от счетов PLS?
2. Почему можно получить красивое разделение на PC1--PC2 и при этом бесполезную модель для y ?

10 Построчный разбор кода

Фрагменты ниже --- точные копии функций из пакета (файлы `preprocess.py`, `pls.py`, `groups.py`, `io.py`). Номера строк внутри каждого листинга свои, с единицы; под листингом каждая строка или группа строк объяснена. Откройте файл в PyCharm рядом: имя функции в заголовке подскажет, где искать.

10.1 Предобработка (`preprocess.py`)

Класс SNV

```

1 class SNV(BaseEstimator, TransformerMixin):
2     """Standard Normal Variate: каждый спектр центрируется и делится на свой SD."""
3
4     def fit(self, X, y=None):
5         return self
6
7     def transform(self, X):
8         X = np.asarray(X, dtype=float)
9         sd = X.std(axis=1, ddof=1, keepdims=True)
10        sd[sd == 0] = 1.0
11        return (X - X.mean(axis=1, keepdims=True)) / sd

```


- 1 Класс-трансформер. BaseEstimator даёт get_params/set_params, TransformerMixin --- метод fit_transform.
- 2 Строка документации.
- 4--5 fit: SNV ничего не «запоминает» (параметры берутся из каждого спектра), поэтому просто возвращает self. Возврат self --- соглашение scikit-learn, благодаря которому работают цепочки вида obj.fit(X).transform(X). Аргумент y=None нужен для совместимости с Pipeline.
- 7 transform: применить преобразование.
- 8 Привести вход (список, DataFrame) к массиву чисел с плавающей точкой.
- 9 SD каждого спектра. axis=1: вдоль длин волн; ddof=1: делим на $p - 1$; keepdims=True: результат формы $(n, 1)$, чтобы дальше работало транслирование.
- 10 Защита: если спектр постоянный ($s = 0$), делитель заменяется на 1, чтобы не получить деление на ноль (результат будет нулевым спектром).
- 11 $(x_{ij} - \bar{x}_i)/s_i$: из (n, p) вычитается $(n, 1)$ (среднее каждой строки), затем деление на $(n, 1)$.

Класс MSC

```

1 class MSC(BaseEstimator, TransformerMixin):
2     """Multiplicative Scatter Correction относительно среднего обучающего спектра."""
3
4     def fit(self, X, y=None):
5         self.reference_ = np.asarray(X, dtype=float).mean(axis=0)
6         return self
7
8     def transform(self, X):
9         X = np.asarray(X, dtype=float)
10        out = np.empty_like(X)
11        for i, row in enumerate(X):
12            slope, intercept = np.polyfit(self.reference_, row, 1)
13            out[i] = (row - intercept) / (slope if slope != 0 else 1.0)
14        return out

```

- 4--6 fit: вычисляет эталон --- средний спектр по обучающим образцам (axis=0, по каждой длине волны). Подчёркивание в reference_ --- признак «появилось после fit».
- 9 Приведение к массиву чисел.
- 10 Пустой массив той же формы для результата.
- 11 Цикл по спектрам; enumerate даёт номер строки i и сам спектр row.
- 12 Подгонка прямой $x_{ij} \approx a_i + b_i x_{\text{ref},j}$: polyfit(x=эталон, y=спектр, степень 1) возвращает [наклон, свободный член], распаковываем в slope, intercept.
- 13 $(x - a)/b$; если наклон 0, делим на 1 (защита).
- 14 Вернуть исправленную матрицу.

Класс SavGol

```

1 class SavGol(BaseEstimator, TransformerMixin):
2     """Сглаживание / производные Савицкого-Голея вдоль спектра."""
3
4     def __init__(self, window=11, polyorder=2, deriv=0):
5         self.window = window
6         self.polyorder = polyorder
7         self.deriv = deriv
8
9     def fit(self, X, y=None):
10         return self
11
12     def transform(self, X):
13         return savgol_filter(np.asarray(X, dtype=float), self.window,
14                               self.polyorder, deriv=self.deriv, axis=1)

```

- 4--7** Конструктор с параметрами по умолчанию (окно 11, полином степени 2, производная 0). Каждый параметр сохраняется *под тем же именем* (`self.window = window`): так требует `BaseEstimator`, иначе `clone` при кросс-валидации не воспроизведёт объект.
- 9--10** `fit` ничего не обучает.
- 13--14** Вызов функции SciPy; `axis=1` означает, что фильтр идёт вдоль каждой строки (вдоль спектра), а не поперёк образцов. Параметр `delta` не задан, поэтому производная считается по номеру точки (см. ловушку в разделе 5.6).

Класс MinMaxSpectrum

```

1 class MinMaxSpectrum(BaseEstimator, TransformerMixin):
2     """Min-max 0-1 ПО КАЖДОМУ СПЕКТРУ (вдоль длин волн). Параметров не обучает."""
3
4     def fit(self, X, y=None):
5         return self
6
7     def transform(self, X):
8         X = np.asarray(X, dtype=float)
9         lo = X.min(axis=1, keepdims=True)
10        rng = X.max(axis=1, keepdims=True) - lo
11        rng[rng == 0] = 1.0
12        return (X - lo) / rng

```

- 9** Минимум каждого спектра, форма $(n, 1)$.
- 10** Размах = максимум минус минимум.
- 11** Если размах 0, заменить на 1 (защита от деления на ноль).
- 12** $(x - \min) / (\max - \min)$.

make_steps и make_preprocessor

```

1 def make_steps(spec) -> list[tuple[str, object]]:
2     """Список шагов из описания вида `["snv", {"savgol": {"deriv": 1}}]`."""
3     steps = []

```

```

4     for i, item in enumerate(spec or []):
5         name, params = (item, {}) if isinstance(item, str) else next(iter(item.items()))
6         if name not in REGISTRY:
7             raise KeyError(f"Неизвестная предобработка '{name}'. Доступны: {sorted(REGISTRY)}")
8         steps.append((f"{i}_{name}", REGISTRY[name](**params)))
9     return steps

```

- 1 Принимает описание цепочки (список), возвращает список пар (имя шага, объект), как требует Pipeline.
- 3 Пустой список для результата.
- 4 `spec or []`: если передали None или пустой список, цикл просто не выполнится. `enumerate` даёт номер шага `i`.
- 5 Элемент списка бывает строкой ("snv", без параметров) или словарём ({"savgol": {"deriv": 1}}). Условное выражение: для строки пара (имя, {}); для словаря `next(iter(item.items()))` берёт первую пару «ключ--значение». Результат распаковывается в `name, params`.
- 6--7 Неизвестное имя: сразу ошибка со списком допустимых имён (`sorted(REGISTRY)`).
- 8 `REGISTRY[name]` --- класс по имени; `(**params)` создаёт его объект с этими параметрами. Имя шага `f"{i}_{name}"` получает номер, чтобы имена были уникальны (две копии `savgol` в одной цепочке).
- 9 Вернуть список шагов.

```

1 def make_preprocessor(spec) -> Pipeline | str:
2     """Pipeline предобработки; для пустой цепочки — ``passthrough``."""
3     steps = make_steps(spec)
4     return Pipeline(steps) if steps else "passthrough"

```

- 3--4 Если шаги есть, собрать Pipeline; иначе строка "passthrough", специальное значение `scikit-learn` «ничего не делать».

10.2 Метрики, кросс-валидация, PLS (pls.py)

rmse и r2

```

1 def rmse(y_true, y_pred) -> float:
2     y_true, y_pred = np.ravel(y_true), np.ravel(y_pred)
3     return float(np.sqrt(np.mean((y_true - y_pred) ** 2)))

```

- 2 `np.ravel` превращает оба входа в одномерные. Это не формальность: если вычесть вектор формы $(n, 1)$ из вектора формы $(n,)$, NumPy растянёт их до матрицы $n \times n$ и результат будет бессмысленным. `PLSRegression.predict` как раз возвращает форму $(n, 1)$.
- 3 $\sqrt{\frac{1}{k} \sum (y - \hat{y})^2}$; `float(...)` превращает число NumPy в обычное число Python.

```

1 def r2(y_true, y_pred) -> float:
2     y_true, y_pred = np.ravel(y_true), np.ravel(y_pred)
3     ss_res = np.sum((y_true - y_pred) ** 2)

```

```

4 ss_tot = np.sum((y_true - y_true.mean()) ** 2)
5 return float(1 - ss_res / ss_tot)

```

- 3 Сумма квадратов остатков (числитель).
- 4 Полная сумма квадратов: отклонения y от своего среднего (знаменатель).
- 5 $1 - SS_{\text{res}}/SS_{\text{tot}}$.

get_cv

```

1 def get_cv(scheme: str):
2     """'loo' | 'logo' | 'groupkfold5' (число – любое)."""
3     s = scheme.lower()
4     if s == "loo":
5         return LeaveOneOut()
6     if s == "logo":
7         return LeaveOneGroupOut()
8     if s.startswith("groupkfold"):
9         return GroupKFold(n_splits=int(s.removeprefix("groupkfold") or 5))
10    raise ValueError(f"Неизвестная схема CV '{scheme}'")

```

- 3 Строка в нижний регистр, чтобы "LOGO" и "logo" были одним и тем же.
- 4--7 По имени возвращается нужный объект-разбиватель.
- 8--9 Для groupkfold5: removeprefix отрезает слово, остаётся "5"; "5" or 5 возьмёт строку, а для пустой строки (просто groupkfold) --- запасное значение 5; int(...) превращает в число.
- 10 Любое другое имя --- ошибка.

build_model

```

1 def build_model(prepare_spec, n_components: int) -> Pipeline:
2     """Предобработка + PLS (центрирование внутри PLS, шкалирование отключено)."""
3     steps = make_steps(prepare_spec)
4     steps.append(("pls", PLSRegression(n_components=n_components, scale=False)))
5     return Pipeline(steps)

```

- 3 Шаги предобработки из описания цепочки.
- 4 Последним шагом добавляется PLS с нужным числом компонент; scale=False: только центрирование.
- 5 Единый объект: fit обучает все шаги по порядку, predict применяет их же.

scan_components --- главная функция

```

1 def scan_components(X, y, groups=None, prepare_spec=None, max_lv: int = 15,
2                     cv: str = "logo") -> pd.DataFrame:
3     """RMSEC / RMSECV / R² для 1..max_lv латентных переменных."""
4     _check_cv_args(cv, groups)

```

```

5 X, y = np.asarray(X, float), np.asarray(y, float)
6 splitter = get_cv(cv)
7 if cv.lower() == "loo":
8     groups = None # LOO группы не использует
9     # n_components не может превышать размер обучающей части минус 1 в любом фолде
10    min_train = min(len(tr) for tr, _ in splitter.split(X, y, groups))
11    max_lv = int(min(max_lv, min_train - 1, X.shape[1]))
12    rows = []
13    for lv in range(1, max_lv + 1):
14        model = build_model(prepare_spec, lv)
15        y_cal = model.fit(X, y).predict(X)
16        y_cv = cross_val_predict(model, X, y, groups=groups, cv=splitter).ravel()
17        rows.append({"LV": lv, "RMSEC": rmse(y, y_cal), "R2C": r2(y, y_cal),
18                    "RMSECV": rmse(y, y_cv), "R2CV": r2(y, y_cv)})
19    return pd.DataFrame(rows)

```

- 1--2 Аргументы: данные, группы, цепочка предобработки, максимум LV, схема CV. Результат --- таблица.
- 4 Проверка: для `logo` и `groupkfold` группы обязательны.
- 5 Приведение к массивам чисел.
- 6 Объект-разбиватель по имени схемы.
- 7--8 LOO группы не использует; чтобы `scikit-learn` не выдавал предупреждения, обнуляем.
- 10 `splitter.split(...)` выдаёт пары (индексы обучения, индексы проверки). Генераторное выражение берёт длину обучающей части в каждом фолде, `min` --- наименьшую.
- 11 Число компонент не может превышать (наименьший размер обучения минус 1) и число переменных. `min` из трёх значений, `int` --- на случай, если пришёл не целый тип.
- 12 Список для строк результата.
- 13 Перебираем число латентных переменных от 1 до максимума.
- 14 Чистая модель с `lv` компонентами.
- 15 **Калибровка:** модель обучается на всех данных и предсказывает те же данные; отсюда `RMSEC` и R_C^2 .
- 16 **Кросс-валидация:** для каждого фолда обучается копия модели на обучающей части и предсказывает проверочную; вектор собирается в исходном порядке; `.ravel()` --- в одномерный вид.
- 17--18 Словарь с четырьмя метриками для этого `lv` добавляется в список.
- 19 Из списка словарей получается таблица: по строке на каждое число LV.

best_lv и benchmark

```

1 def best_lv(scan: pd.DataFrame) -> int:
2     """LV с минимальным RMSECV."""
3     return int(scan.loc[scan["RMSECV"].idxmin()], "LV")

```

- 3 `idxmin()` --- метка строки с минимальным `RMSECV`; `.loc[метка, "LV"]` --- число LV в этой строке; `int` --- в целое.

```

1 def benchmark(X, y, groups, chains: dict, max_lv: int = 15, cv: str = "logo") -> pd.DataFrame:
2     """Сравнение цепочек предобработки: по строке на цепочку при лучшем LV."""
3     out = []
4     for label, spec in chains.items():
5         scan = scan_components(X, y, groups, spec, max_lv=max_lv, cv=cv)
6         row = scan.loc[scan["RMSECV"].idxmin()].to_dict()
7         row["LV"] = int(row["LV"])
8         out.append({"preprocessing": label, **row})
9     return pd.DataFrame(out).sort_values("RMSECV").reset_index(drop=True)

```

- 4 Цикл по словарю «название цепочки → описание». `.items()` даёт пары.
- 5 Для каждой цепочки --- полный перебор числа LV.
- 6 Строка с лучшим RMSECV, превращённая в словарь.
- 7 Pandas хранит строку как числа с плавающей точкой, поэтому LV снова делаем целым.
- 8 `{"preprocessing": label, **row}` --- словарь, в котором к названию добавлены все пары из `row`.
- 9 Таблица, отсортированная по возрастанию RMSECV (лучшая цепочка сверху); `reset_index(drop=True)` перенумеровывает строки с нуля и отбрасывает старые номера.

10.3 Группы (`groups.py`)

```

1 def infer_groups(names: Sequence[str]) -> list[str]:
2     """Определяет серию по имени образца (эвристика, ОБЯЗАТЕЛЬНО проверять глазами).
3
4     Правила: если в имени есть ``dry tissue`` / ``soft tissue`` — группа
5     «<тип>_<первое чисто числовое поле>»; иначе группа «series_<первое чисто
6     числовое поле>». Поля разделяются ``_``. Если числа нет — образец
7     становится отдельной группой.
8     """
9     out = []
10    for i, name in enumerate(names):
11        tokens = str(name).split("_")
12        kind = next((k for k in ("dry", "soft") if tokens[0].lower().startswith(k)), None)
13        number = next((t for t in tokens[1:] if re.fullmatch(r"\d+", t)), None)
14        if kind is None:
15            number = next((t for t in tokens if re.fullmatch(r"\d+", t)), None)
16        prefix = kind if kind else "series"
17        out.append(f"{prefix}_{number}" if number else f"single_{i}")
18    return out

```

- 1 Принимает имена образцов, возвращает список названий групп той же длины.
- 9 Список результата.
- 10 Номер образца `i` нужен для имён «одинокых» групп.
- 11 `str(name).split("_)` режет имя по подчёркиваниям: `"first_10" → ["first", "10"]`.
- 12 Ищем тип ткани: первый из `"dry"`, `"soft"`, с которого начинается первое поле (регистр не важен); иначе `None`.

- 13 Номер серии: первое поле *целиком из цифр*, считая со второго (`tokens[1:]`). Поэтому у "dry tissue_first_1_0.069_..." номер 1, а "0.069" не подходит (есть точка).
- 14--15 Для имён без типа ткани ("first_10", "9_first") цифровое поле ищем во всех полях, включая первое.
- 16 Префикс группы: тип ткани или слово `series`.
- 17 Название группы: "dry_1" и т.п., а если числа нет --- "single_<номер>" (образец сам по себе).

10.4 Чтение и склейка файлов (`io.py`)

`_parse_two_columns`

```

1 def _parse_two_columns(text: str) -> tuple[np.ndarray, np.ndarray]:
2     """Первые два числа в каждой строке; нечисловые строки (шапка) пропускаются.
3
4     Понимает разделители «пробел / табуляция / ;» и десятичную запятую.
5     """
6     xs, ys = [], []
7     for line in text.splitlines():
8         line = line.strip()
9         if not line:
10             continue
11         if re.search(r"[;\t ]", line):
12             tokens = [t.replace(",", ".") for t in re.split(r"[;\t ]+", line) if t]
13         else: # csv с запятой-разделителем и точкой-десятичным знаком
14             tokens = line.split(",")
15         try:
16             xs.append(float(tokens[0]))
17             ys.append(float(tokens[1]))
18         except (ValueError, IndexError):
19             continue
20     if not xs:
21         raise ValueError("Не найдено ни одной строки с двумя числами")
22     return np.asarray(xs), np.asarray(ys)

```

- 6 Списки для длин волн и поглощений.
- 7--8 Перебор строк текста; `strip()` убирает пробелы по краям.
- 9--10 Пустые строки пропускаются.
- 11--12 Если в строке есть точка с запятой, табуляция или пробел --- режим по любой серии таких символов; в каждом куске запятую заменяем на точку (десятичная запятая); `if t` отбрасывает пустые куски.
- 13--14 Иначе считаем строку записью через запятую (csv), когда десятичный знак --- точка.
- 15--19 Пробуем превратить первые два куска в числа. Строка заголовка (`Wavelength;Abs`) вызовет `ValueError`, строка с одним числом --- `IndexError`; в обоих случаях `continue` пропускает строку.
- 20--21 Если не найдено ни одной числовой строки --- ошибка.
- 22 Два массива NumPy.

merge_txt_files

```

1 def merge_txt_files(paths: Sequence, on_mismatch: str = "interpolate",
2                     decimals: int = 6) -> pd.DataFrame:
3     """Склейка txt-спектров в одну таблицу: ``wavelength`` + столбец на образец.
4
5     Имя столбца — имя файла без расширения. Если сетки длин волн у файлов
6     различаются: ``on_mismatch='interpolate'`` — линейная интерполяция на сетку
7     первого файла (вне перекрытия — NaN), ``'raise'`` — ошибка.
8     """
9     if on_mismatch not in {"interpolate", "raise"}:
10         raise ValueError("on_mismatch: 'interpolate' или 'raise'")
11     paths = [Path(p) for p in paths]
12     if not paths:
13         raise ValueError("Пустой список файлов")
14     ref_wl, first = read_txt_spectrum(paths[0])
15     cols = {paths[0].stem: first}
16     mismatched = []
17     for p in paths[1:]:
18         wl, a = read_txt_spectrum(p)
19         if wl.shape == ref_wl.shape and np.allclose(wl, ref_wl, atol=10 ** -decimals):
20             cols[p.stem] = a
21             continue
22         if on_mismatch == "raise":
23             raise ValueError(f"Сетка длин волн в {p.name} не совпадает с {paths[0].name}")
24         order = np.argsort(wl)
25         cols[p.stem] = np.interp(ref_wl, wl[order], a[order], left=np.nan, right=np.nan)
26         mismatched.append(p.name)
27     if mismatched:
28         warnings.warn(f"Интерполяция на сетку {paths[0].name} для: {'', '.join(mismatched)}")
29     if len(cols) != len(paths):
30         raise ValueError("Имена файлов (без расширения) должны быть уникальны")
31     return pd.DataFrame({"wavelength": ref_wl, **cols})

```

9--10 Допустимы только два режима при несовпадении сеток.

11 Все пути в объекты Path.

12--13 Пустой список файлов --- ошибка.

14 Первый файл задаёт эталонную сетку длин волн.

15 Словарь «имя файла без расширения → значения поглощения».

16 Список файлов, у которых сетка отличалась.

17--18 Остальные файлы читаем по одному.

19--21 Если сетка той же формы и совпадает с допуском 10^{-6} --- берём значения как есть и переходим к следующему файлу.

22--23 Режим raise: несовпадение --- ошибка.

24 np.interp требует возрастающей сетки, поэтому сортируем: argsort даёт нужный порядок индексов.

25 Линейная интерполяция на эталонную сетку; вне диапазона файла --- NaN (left, right).

26 Запоминаем имя для предупреждения.

27--28 Если были несовпадения, выдаём предупреждение со списком.

- 29--30 Два файла с одинаковым именем (без расширения) перезаписали бы друг друга в словаре; тогда столбцов меньше, чем файлов --- ошибка.
- 31 Таблица: столбец `wavelength` и по столбцу на образец; `**cols` вставляет пары словаря.

read_samples_xlsx

```

1 def read_samples_xlsx(path, sheet=0, name_col=0, y_col=1, header_row=0) -> Dataset:
2     """Читает Excel: имя образца, Y, затем спектр; в строке-шапке – длины волн."""
3     raw = pd.read_excel(path, sheet_name=sheet, header=None)
4     first_spec = max(name_col, y_col) + 1
5     wl = pd.to_numeric(raw.iloc[header_row, first_spec:], errors="coerce")
6     if wl.isna().any():
7         raise ValueError("В строке-шапке есть нечисловые длины волн")
8     body = raw.drop(index=header_row)
9     X = body.iloc[:, first_spec:].apply(pd.to_numeric, errors="coerce")
10    y = pd.to_numeric(body.iloc[:, y_col], errors="coerce")
11    if X.isna().any().any() or y.isna().any():
12        raise ValueError("Найдены пропуски или нечисловые значения в X или Y")
13    return Dataset(
14        X=X.to_numpy(), wavelengths=wl.to_numpy(),
15        names=body.iloc[:, name_col].astype(str).tolist(), y=y.to_numpy(),
16    )

```

- 3 Чтение листа без заголовка (всё как данные).
- 4 Номер первого столбца спектра: после столбцов имени и Y ($\max(0, 1) + 1 = 2$).
- 5 Первая строка, столбцы спектра → числа (длины волн); то, что не число, станет NaN.
- 6--7 Если в шапке есть не числа --- ошибка.
- 8 Данные без строки шапки.
- 9 Матрица спектров в числа.
- 10 Столбец Y в числа.
- 11--12 `X.isna().any().any()` --- есть ли хоть один пропуск в таблице (первый `any` по столбцам, второй по результату). При пропуске --- ошибка, а не тихая подстановка нуля.
- 13--16 Сборка объекта `Dataset`; `to_numpy()` превращает таблицы в массивы NumPy, имена --- в список строк.

11 Ответы на вопросы для самопроверки

Раздел 2 (Python). (1) Вектор длины 80 --- среднее по спектру каждого образца; с `keepdims=True` форма (80, 1). (2) Числа с плавающей точкой округляются, и результат вычислений может отличаться от «точного» в последних знаках; `allclose` сравнивает с допуском. (3) `clone`, который вызывает `scikit-learn` в кросс-валидации, создаёт новый объект по `get_params`, а он читает атрибуты с теми же именами, что и параметры конструктора; несовпадение даёт ошибку или потерю заданного значения. (4) `loc` --- по подписям (метки индекса и столбцов), `iloc` --- по целочисленным позициям.

Раздел 3 (математика). (1) 465×465 ; ранг не больше 79 (после центрирования, при $n = 80$). (2) Вычесть средний спектр; среднее считается по образцам, $\text{axis}=0$. (3) При $\text{ddof} = 1$ делим на меньшее число ($n - 1$), и результат больше.

Раздел 4 (спектры). (1) Поглощение смеси есть сумма вкладов компонентов, пропорциональных концентрациям: спектр линейно зависит от концентраций, что и описывает линейная модель. (2) Измерение --- один замер, серия --- один физический образец на разных стадиях сушки; соседние точки серии почти одинаковы, и если их разнести между обучением и проверкой, модель «узнает» кусок. (3) Абсолютная интенсивность теряется, остаётся только форма спектра.

Раздел 5 (предобработка). (1) SNV использует только сам спектр, MSC использует эталон из обучающих образцов. (2) Первый нормирует каждый спектр по его размаху, второй --- каждый столбец по образцам; в вашем файле выполнена нормировка по спектру на максимум. (3) Среднее 4, отклонения $(-2, 0, 2)$, сумма квадратов 8, $s^2 = 8/2 = 4$, $s = 2$; результат $(-1, 0, 1)$. (4) Убирает аддитивный сдвиг и подчёркивает форму полос, но усиливает шум.

Раздел 6 (PLS). (1) Ковариацию счёта $\mathbf{X}\mathbf{w}$ с $\mathbf{y}$ (PCA максимизирует только дисперсию $\mathbf{X}$). (2) После вычитания $\mathbf{X}^{(a+1)\top} \mathbf{t}_a = 0$, а $\mathbf{t}_{a+1}$ --- комбинация столбцов $\mathbf{X}^{(a+1)}$, поэтому $\mathbf{t}_a^\top \mathbf{t}_{a+1} = 0$. (3) RMSEC убывает; RMSECV сначала убывает, потом растёт. (4) Какие длины волн и с каким знаком влияют на предсказание.

Раздел 7 (валидация). (1) Модель оценивается на тех же данных, на которых обучалась. (2) $R^2 = 0$. (3) Соседи по серии в обучении подсказывают ответ. (4) Эффект отбора: выбран лучший из огромного числа вариантов, и его ошибка оптимистична.

Раздел 8 (интервалы и GA). (1) $\mathbf{W} = \begin{pmatrix} 1/2 & 0 \\ 1/2 & 1/2 \\ 0 & 1/2 \end{pmatrix}$, $z_{i1} = (x_{i1} + x_{i2})/2$, $z_{i2} = (x_{i2} + x_{i3})/2$.
(2) $\text{rank}(\mathbf{X}\mathbf{W}) \leq \text{rank } \mathbf{X}$, новых направлений не возникает. (3) Это быстрее (одна подгонка вместо многих). (4) Сохраняют: отбор и элитизм; разнообразие добавляют: мутация и скрещивание.

Раздел 9 (PCA). (1) Счета PCA выбираются только по дисперсии $\mathbf{X}$, счета PLS --- с учётом связи с $\mathbf{y}$. (2) Главная изменчивость (например, разбавление, рассеяние) может не иметь отношения к $\mathbf{y}$.

12 Глоссарий

Русский	English	Смысл
латентная переменная	latent variable, LV	новая переменная, построенная из исходных, в PLS
счета	scores	координаты образцов в латентном пространстве
нагрузки	loadings	вклад исходных переменных в латентную
калибровка	calibration	построение модели по образцам с известным y
валидация	validation	проверка модели на образцах, не участвовавших в обучении
кросс-валидация	cross-validation, CV	многократное разбиение на обучение и проверку
утечка данных	data leakage	информация из проверочной выборки попала в обучение
предобработка	preprocessing	преобразование спектров перед моделью
понижение разрешения	de-resolution	усреднение по интервалам
генетический алгоритм	genetic algorithm, GA	эволюционный метод оптимизации
приспособленность	fitness	целевая функция особи
выброс	outlier	образец, сильно отличающийся от остальных
сбалансированная	balanced accuracy	среднее чувствительности и специфичности
точность		

13 Литература

1. Galyanin V., Melenteva A., Bogomolov A. Selecting optimal wavelength intervals for an optical sensor: A case study of milk fat and total protein analysis in the region 400--1100 nm. *Sensors and Actuators B* 218 (2015) 97--104.
2. Latysheva A. et al. Near-infrared spectroscopy for urine-based cancer detection: a systematic characterization of the analytical challenges of a complex biofluid matrix. *Spectrochimica Acta Part A* 365 (2027) 128663.
3. Wold S., Sjöström M., Eriksson L. PLS-regression: a basic tool of chemometrics. *Chemom. Intell. Lab. Syst.* 58 (2001) 109--130.
4. Rinnan Å., van den Berg F., Engelsen S.B. Review of the most common pre-processing techniques for near-infrared spectra. *TrAC Trends Anal. Chem.* 28 (2009) 1201--1222.
5. Barnes R.J., Dhanoa M.S., Lister S.J. Standard normal variate transformation and de-trending of near-infrared diffuse reflectance spectra. *Appl. Spectrosc.* 43 (1989) 772--777.
6. Geladi P., MacDougall D., Martens H. Linearization and scatter-correction for near-infrared reflectance spectra of meat. *Appl. Spectrosc.* 39 (1985) 491--500.
7. Savitzky A., Golay M.J.E. Smoothing and differentiation of data by simplified least squares procedures. *Anal. Chem.* 36 (1964) 1627--1639.
8. Hastie T., Tibshirani R., Friedman J. *The Elements of Statistical Learning*, 2nd ed. Springer, 2009 (кросс-валидация, глава 7).
9. Pedregosa F. et al. Scikit-learn: machine learning in Python. *J. Mach. Learn. Res.* 12 (2011) 2825--2830.

Статьи 1 и 2 прочитаны по присланным вам PDF; выходные данные источников 3--9 приведены по памяти, перед цитированием в работе сверьте их с оригиналами.